JGI JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Spring Boot Tomcat Custom Port Number and Spring Boot Rest Template

Table of Contents

UNIT 4: Spring Boot Tomcat Custom Port Number and Spring Boot Rest Template

Objectives

In this Unit you will learn –

- To set a custom port number for your Spring Boot application
- To set a random port number for your Spring Boot application
- The differences between SOAP and REST
- The basics of HTTP methods: GET, POST, PUT, DELETE
- To consume RESTful web services using Rest Template
- To handle errors in Rest Template

Learning Outcome:

Understand Spring Boot's Port Number and REST template

4. Spring Boot Tomcat Custom Port Number and Spring Boot Rest Template

1. Spring Boot Port Number

In Spring Boot, the port number is used to specify the port on which the application will run. By default, Spring Boot applications use port number 8080, but you can change this to any other port number.

Custom Port Number

Custom port number is a port number that you can specify manually to run your Spring Boot application on a specific port number. This is useful if you want to run multiple applications on the same server with different port numbers.

Random Port Number

Random port number is a port number that is automatically assigned by Spring Boot when the application starts up. This is useful if you want to run your application on a random port number, which can help you avoid port conflicts when running multiple instances of your application.

How to set a custom port number

To set a custom port number in Spring Boot, you can add the following line to your application.properties file:

```
server.port=8081
```

This will set the port number to 8081. You can replace 8081 with any other port number of your choice.

You can also set the port number programmatically using the following code:

```
@SpringBootApplication

public class MyApplication {

  public static void main(String[] args) {

    SpringApplication app = new SpringApplication(MyApplication.class);

            app.setDefaultProperties(Collections.singletonMap("server.port",
 "8081"));

    app.run(args);

  }

}
```

This will set the port number to 8081.

How to set a random port number

To set a random port number in Spring Boot, you can use the following line in your application.properties file:

server.port=0

This will tell Spring Boot to assign a random port number to your application when it starts up. You can retrieve the assigned port number programmatically using the following code:

```
@Value("${server.port}")

int port;
```

This will inject the assigned port number into the port variable.

How to configure multiple ports in Spring Boot:

To configure multiple ports in Spring Boot, you can add the following lines to your application.properties file:

```
server.port=8080, 8081
```

This will configure your application to run on both port 8080 and 8081. You can also specify a range of port numbers using the following line:

```
server.port=8080-8083
```

This will configure your application to run on port numbers 8080, 8081, 8082, and 8083.

Note that when you configure multiple ports, the application will run on all specified ports simultaneously. If you want to run the application on only one port, you should specify a single port number.

2. Spring Boot Rest Template

Rest Template is a class in the Spring Framework that provides a convenient way to consume RESTful web services. Rest Template simplifies the process of making HTTP requests and processing HTTP responses, and it supports a wide range of HTTP methods and response formats.

Advantages of using Rest Template

- Rest Template simplifies the process of making HTTP requests and processing HTTP responses.

- Rest Template provides a consistent interface for consuming RESTful web services, regardless of the underlying implementation.
- Rest Template supports a wide range of HTTP methods and response formats, making it a versatile tool for consuming web services.
- Rest Template integrates well with other Spring Framework components, making it easy to use in Spring Boot applications.

Differences between SOAP and REST

SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) are two popular web service architectures used for exchanging data between applications over the internet. Here are some differences between SOAP and REST:

1. **Architecture:** SOAP is a protocol-based architecture that relies on XML messages to communicate between client and server. REST, on the other hand, is a resource-based architecture that uses a URL and HTTP methods to communicate between client and server.

2. **Data format:** SOAP uses XML to encode data, whereas REST can use different data formats such as JSON, XML, or plain text.

3. **Message structure:** SOAP messages have a predefined structure with an envelope, header, and body. REST messages do not have a predefined structure, and the message format is usually determined by the data format used.

4. **Transport:** SOAP can use different transport protocols such as HTTP, SMTP, or TCP. REST uses only HTTP as a transport protocol.

5. **Caching:** REST allows for caching of responses, which can improve performance.

SOAP does not provide any caching mechanism.

6. **Security:** SOAP has built-in support for security features such as encryption and digital signatures. REST relies on HTTPS for secure communication.

7. **Performance:** REST is generally considered to be faster and more lightweight than SOAP, as it does not require as much processing and overhead.

SOAP is a protocol-based architecture that relies on XML messages, while REST is a resource-based architecture that uses a URL and HTTP methods. SOAP has a predefined message structure, while REST does not. REST is generally considered faster and more lightweight than SOAP, but SOAP has built-in support for security features.

Understanding HTTP Methods

HTTP methods are the verbs that define the actions that can be performed on a resource over the web. Here are some of the most commonly used HTTP methods:

1. **GET:** This method is used to retrieve data from a specified resource. It should not modify any data on the server and should only retrieve data.

2. **POST:** This method is used to submit an entity to a specified resource, often causing a change in state on the server. It can be used for creating or updating resources.

3. **PUT:** This method is used to replace all current representations of a resource with the entity provided in the request. It is typically used for updating a resource.

4. DELETE: This method is used to delete a specified resource. It should not have any side effects on the server other than the removal of the specified resource.

Here's a more detailed explanation of each method:

GET

- When a client makes a GET request to a server, it is asking the server to retrieve data from a specific resource.

- The data returned by the server is typically in the form of a response body, which may be in a variety of formats, such as HTML, XML, or JSON.

- GET requests are considered safe and idempotent, meaning that they should not modify any data on the server and can be repeated multiple times without causing any side effects.

POST

- When a client makes a POST request to a server, it is asking the server to accept an entity and store it at a specific resource.

- The entity is typically included in the request body, and the server may return a response that includes information about the new resource.

- POST requests are not idempotent, meaning that they may cause a change in state on the server each time they are executed.

PUT

- When a client makes a PUT request to a server, it is asking the server to replace all current representations of a resource with the entity provided in the request.

- The entity is typically included in the request body, and the server may return a response that includes information about the updated resource.

- PUT requests are idempotent, meaning that they can be repeated multiple times

without causing any side effects.

DELETE

- When a client makes a DELETE request to a server, it is asking the server to delete a specific resource.
- The server may return a response indicating the success or failure of the operation.
- DELETE requests are idempotent, meaning that they can be repeated multiple times without causing any side effects.

Each HTTP method has a specific purpose and is used for different types of operations on resources over the web. Rest Template provides a simple way to consume RESTful web services using any of these methods.

How to consume RESTful web services using Rest Template

Consuming RESTful web services using Rest Template is a straightforward process. Here are the general steps involved:

1. Create a Rest Template instance: The first step is to create an instance of the Rest Template class. This can be done using the default constructor or by using a builder.

2. Define the URL: The next step is to define the URL of the RESTful web service you want to consume. This can be done using a string or a URI object.

3. Define the HTTP method: The next step is to define the HTTP method you want to use to interact with the web service. This can be done using one of the methods provided by Rest Template, such as getForObject(), postForObject(), put(), delete(), etc.

4. Define the request headers and body: If necessary, you can define the request headers and body using the methods provided by Rest Template, such as headers() and body().

5. Execute the request: Once you have defined the URL, HTTP method, request headers, and body, you can execute the request using the Rest Template method. The method will return the response from the web service, which can be processed according to your needs.

Here's a sample code snippet that demonstrates how to consume a RESTful web service using Rest Template:

```
RestTemplate restTemplate = new RestTemplate();

String url = "https://jsonplaceholder.typicode.com/posts/1";
Post post = restTemplate.getForObject(url, Post.class);

System.out.println("Post title: " + post.getTitle());
```

In this example, we create an instance of Rest Template and define the URL of a RESTful web service that returns a Post object. We then use the getForObject() method to execute the request and retrieve the response as a Post object. Finally, we print the title of the post to the console.

Consuming RESTful web services using Rest Template is a simple process that involves creating an instance of the Rest Template class, defining the URL and HTTP method, and executing the request. Rest Template provides a convenient way to work with RESTful web services in Spring Boot applications.

How to handle errors in Rest Template

When consuming RESTful web services using Rest Template, it's important to handle errors that may occur during the request. Rest Template provides several methods to handle errors, including:

1. **Exception handling:** Rest Template throws several exceptions in case of errors, such as HttpClientErrorException, HttpServerErrorException, ResourceAccessException, etc. You can catch these exceptions using a try-catch block and handle them accordingly.

For example:

```
RestTemplate restTemplate = new RestTemplate();
String url = "https://jsonplaceholder.typicode.com/posts/1000";

try {
    Post post = restTemplate.getForObject(url, Post.class);
    System.out.println("Post title: " + post.getTitle());
} catch (HttpClientErrorException ex) {
    if (ex.getStatusCode() == HttpStatus.NOT_FOUND) {
        System.out.println("Post not found");
    }
} catch (HttpServerErrorException ex) {
    System.out.println("Internal server error: " + ex.getStatusCode());
} catch (ResourceAccessException ex) {
    System.out.println("Error connecting to the server: " +
 ex.getMessage());
}
```

In this example, we try to get a Post object from a RESTful web service that doesn't exist, which will result in a 404 Not Found error. We catch the HttpClientErrorException exception and check its status code to determine the type of error. We then print an appropriate message to the console.

2. Response error handling: Rest Template also provides a ResponseErrorHandler interface that you can use to handle errors in the response. You can create a custom implementation of the ResponseErrorHandler interface and set it on the Rest Template instance using the setErrorHandler() method.

For example:

```
public class MyResponseErrorHandler implements ResponseErrorHandler {


    @Override
    public boolean hasError(ClientHttpResponse response) throws IOException
 {
        HttpStatus statusCode = response.getStatusCode();
        return (statusCode.series() == HttpStatus.Series.CLIENT_ERROR
                || statusCode.series() == HttpStatus.Series.SERVER_ERROR);
    }


    @Override
    public void handleError(ClientHttpResponse response) throws IOException
 {
        HttpStatus statusCode = response.getStatusCode();
        if (statusCode.series() == HttpStatus.Series.SERVER_ERROR) {
            throw new HttpServerErrorException(statusCode);
        } else if (statusCode.series() == HttpStatus.Series.CLIENT_ERROR) {
```

```
            throw new HttpClientErrorException(statusCode);
        }
    }

}

RestTemplate restTemplate = new RestTemplate();
restTemplate.setErrorHandler(new MyResponseErrorHandler());
```

In this example, we create a custom implementation of the ResponseErrorHandler interface that checks the status code of the response and throws the appropriate exception. We then set this custom error handler on the Rest Template instance using the setErrorHandler() method.

Handling errors in Rest Template involves either catching exceptions or using a custom implementation of the ResponseErrorHandler interface. Rest Template provides several methods to handle errors, making it a robust and reliable tool for consuming RESTful web services.